

Event-First Operational Ontology: Cryptographic Event Chains for Multi-Agent Concept Consensus

Abstract. The proliferation of large language model (LLM) based autonomous agents has intensified interest in multi-agent systems capable of collaborative knowledge synthesis, yet the conceptual outputs generated by such systems lack typed relations, lifecycle traceability, and machine-checkable provenance. Existing operational ontologies address this gap through object-first designs that impose substantial deployment friction, require specialized authoring expertise, and store mutable status fields vulnerable to inconsistency. This paper presents ADL Lite, an event-first operational ontology in which concepts are modeled as append-only, cryptographically hashed EventChains and all lifecycle state—status, confidence, validator identity—is derived exclusively from event history rather than stored as mutable fields. Six experiments establish correctness properties across chain integrity (100% precision and recall on 50 valid and 10 corrupted chains), status derivation (2,204/2,204 correct), snapshot round-trip consistency (38/38 status match), precondition enforcement ($P=1.0$, $R=1.0$, $F1=1.0$ across 13 test cases), multi-agent auditability (5/5 chains verified), and scalability (495,671 chains comprising 5,080,714 events processed in 238 seconds with zero integrity failures). ADL Lite is pip-installable, operates over Markdown files in standard Git repositories, and requires no triple store, OWL reasoner, or enterprise infrastructure, achieving enterprise-grade lifecycle traceability and multi-agent auditability at a deployment cost closer to personal note-taking software than to enterprise knowledge graph platforms.

Keywords: ontology engineering · event sourcing · multi-agent systems · knowledge graphs · anti-money laundering

1 Introduction

1.1 Background and Motivation

The proliferation of large language model (LLM) based autonomous agents has intensified interest in multi-agent systems capable of collaborative discovery and knowledge synthesis [3,41]. Contemporary frameworks such as LangChain, LlamaIndex, and MetaGPT enable agents to decompose complex tasks, invoke external tools, and maintain conversational state across extended interactions. When multiple such agents operate concurrently—each generating hypotheses, evaluating evidence, and proposing conceptual structures—their outputs accumulate as unstructured natural language prose. This unconstrained textual output, while

expressive in semantic range, lacks typed relations, explicit lifecycle tracking, and machine-checkable provenance. The resulting conceptual artifacts are difficult to validate, version, or compose into shared ontological commitments [23,1]. Cross-references between concepts remain implicit in natural language, temporal evolution is undocumented, and the absence of a shared schema prevents agents from building upon one another’s discoveries in a structured fashion. Each agent may develop its own vocabulary, classification criteria, and confidence calibration, leading to semantic fragmentation that undermines collective intelligence.

Traditional ontology engineering addresses this problem through formal specification: domain experts construct object hierarchies, define property constraints, and establish inference rules using languages such as the Web Ontology Language (OWL) [38]. Palantir’s Foundry Data Engine (FDE) exemplifies the industrial state of the art, organizing enterprise knowledge around Object Types, Link Types, Property Types, and Action Types configured through a dedicated user interface [29]. This approach produces rigorously structured ontologies but imposes substantial deployment friction. Ontology construction requires specialized expertise in both the target domain and the formalism, dedicated tooling that may carry significant licensing costs, and manual curation workflows that do not scale to the velocity of agent-generated conceptual output [15]. A further limitation is that LLM-based agents cannot meaningfully participate in this authoring process: they lack programmatic access to ontology construction interfaces and cannot manipulate the underlying formalisms through standard software development kits [30]. The Ontology Manager UI in Palantir FDE, for instance, requires human operators to define object types, link types, and action types through point-and-click configuration; no programmatic creation APIs are exposed for automated agent authorship [30]. Semantic elements (Object Type, Property Type, Link Type) and kinetic elements (Action Type, Function) must be co-defined through this interface, creating a bottleneck that prevents LLM agents from autonomously extending the ontology as they discover new concepts.

The philosophical foundations of this work draw upon Ludwig Wittgenstein’s *Tractatus Logico-Philosophicus* [40]. In proposition §1.1, Wittgenstein asserts: “*The world is the totality of facts, not of things.*” The original German—“*Die Welt ist die Gesamtheit der Tatsachen, nicht der Dinge*”—emphasizes that reality consists not of isolated objects but of states of affairs in which those objects participate [39]. An object, on this view, has no ontological standing independent of the facts in which it appears. A thing gains its significance only through participation in states of affairs that constitute the fabric of the world. The concept of a validated scientific hypothesis, for instance, is not a static entity possessing an intrinsic property of “validatedness”; rather, it is constituted by the totality of validation events, evidentiary assertions, and review actions that have occurred in its history. This insight has direct architectural implications: if concepts are understood not as self-subsistent entities but as loci of event histories, then an ontology should be organized around *actions* rather than *objects*. Status, confidence, and relational embeddings become derivative properties computed from the sequence of events that constitute a concept’s history, not mutable fields

stored alongside the concept itself [13,17]. The event-first stance does not deny the existence of objects—Wittgenstein explicitly acknowledges that objects are constituents of facts—but it denies that objects have ontological priority. In engineering terms, this means that the action registry and event schema are designed first, and object-like abstractions emerge as projections of event patterns rather than as foundational data structures. The event-first architecture thus operationalizes Wittgenstein’s insight: the ontology records what has happened (the totality of facts), not what purportedly exists (a catalog of things), and concepts acquire their identity through the events in which they participate.

1.2 Research Gap and Problem Statement

Existing operational ontologies, including Palantir FDE and OWL-based knowledge graphs, are predominantly *object-first*: an Object Type defines what entities exist, Property Type describes their attributes, Link Type connects them, and Action Type is added as an afterthought to model operations upon the previously defined object structures [29,14]. This design creates deployment bottlenecks at multiple stages of the ontology lifecycle. Concept instances carry mutable status fields that can diverge from their actual event histories when updates occur outside the governed action pathway. Updates are destructive: modifying a concept’s properties overwrites prior state, making audit reconstruction dependent on external logging infrastructure that may itself be incomplete or inconsistent. Agents wishing to participate in ontology evolution must navigate complex user interfaces or specialized SDKs with steep learning curves, precluding lightweight automation at the scale required by modern multi-agent research pipelines [30]. The cumulative effect is that ontology engineering remains a human-intensive activity disconnected from the generative processes that most need semantic structure—the autonomous production of conceptual content by cooperating agents.

The specific problem addressed in this paper is formulated as follows: *How can an operational ontology achieve enterprise-grade lifecycle traceability, cryptographic integrity, and multi-agent consensus without requiring heavyweight tooling, expert curation, or object-first mutable state?* This problem statement motivates the design of a system that is simultaneously (i) pip-installable and Git-backed, enabling frictionless deployment in standard Python environments without enterprise infrastructure; (ii) natively authorable by LLM agents through Markdown files that serve as both human-readable documentation and machine-parseable semantic assertions, eliminating the need for specialized ontology authoring interfaces; (iii) cryptographically auditable via append-only event chains that detect any tampering with the ontological record through SHA-256 hash verification; and (iv) semantically governed by a closed action registry with structured precondition validation that eliminates runtime code injection through statically typed comparator enumeration. The system targets the gap between unstructured agent output and formal ontology deployment, where lightweight operational semantics can accelerate conceptual convergence without claiming to replace enterprise platforms for high-assurance compliance workloads [2].

1.3 Contributions

This paper presents ADL Lite, an event-first operational ontology for multi-agent concept consensus. The system represents concepts as append-only, cryptographically hashed EventChains, with lifecycle state derived exclusively from event history. It is evaluated through six experiments (E1–E6) testing chain integrity, status derivation, snapshot round-trip consistency, precondition enforcement, multi-agent auditability, and scalability on real-world financial transaction data. The four principal contributions are enumerated below.

Contribution 1: Event-first ontology architecture. This paper proposes EventChain as the fundamental representational unit for ontological concepts. Each concept is modeled not as a mutable object with stored properties but as an append-only, cryptographically hashed sequence of typed events. A concept’s status, confidence score, and validator identity are *computed* from the event chain rather than stored as mutable fields. SHA-256 hashing and cryptographic linking via `previous_event_id` ensure that any tampering with the chain is detectable through `verify_integrity()`. The event model includes nine event types spanning the lifecycle (REGISTER, VALIDATE, DEPRECATE, FORK, ARCHIVE), assertions (RELATE, EVIDENCE, SEAL), and communication (ANNOUNCE, PUBLISH). Empirical evaluation on 495,671 EventChains derived from the IBM Anti-Money Laundering (AML) HI-Small dataset demonstrates zero integrity failures across 5,080,714 events processed in 238 seconds (E6), confirming that the architecture scales to production-level data volumes without degradation of cryptographic guarantees.

Contribution 2: L4 action blocks with structured precondition validation. This paper introduces a closed action registry comprising nine core actions (register, validate, fork, deprecate, archive, announce, publish, sync_dashboard, listen), each defined in `adl_core_ontology.yaml` with declarative precondition rules. Precondition validation employs a `Comparator` enumeration (EQ, NEQ, GT, GTE, LT, LTE, IN, EXISTS) evaluated against `ADLFrontMatter` fields through statically typed Pydantic models—eliminating runtime `eval()` entirely. The `ActionExecutor` validates preconditions at parse time before dispatching side effects, ensuring that no action can transition a concept into an invalid state. Thirteen test cases covering all registered actions achieve perfect precision ($P = 1.0$), perfect recall ($R = 1.0$), and perfect F1 score ($F1 = 1.0$), with zero false positives and zero false negatives (E4).

Contribution 3: Empirical validation at scale. This paper reports the results of six experiments establishing correctness properties across the full system: 100% chain integrity verification on 50 valid and 10 intentionally corrupted chains (E1); 100% status derivation accuracy across 2,204 exhaustive event sequences including 2,197 three-event combinations and 7 edge cases (E2); 100% snapshot round-trip consistency across 38 concept files spanning synthetic and real AML domains (E3); 100% precondition enforcement precision and recall on 13 action-validation test cases (E4); 100% multi-agent auditability across 5-agent simulations producing independently verifiable chains (E5); and 100% chain integrity at scale on 495,671 chains from real IBM AML transaction data (E6).

The aggregate evidence supports the claim that event-first derivation of ontological state is computationally feasible, cryptographically sound, and empirically correct under exhaustive test conditions.

Contribution 4: Open-source reference implementation. This paper accompanies the release of `adl_lite`, a Python toolkit providing an `ADLParser` for L1–L4 document parsing with Pydantic validation, an `ActionExecutor` for precondition-validated action dispatch, a `ConsensusEngine` for status transition enforcement with fork semantics, a `DataImporter` for bulk `EventChain` construction from CSV datasets, and a unified experiment runner enabling reproducible evaluation via a single command-line interface. The toolkit is pip-installable, operates over Markdown files stored in standard Git repositories, and requires no external triple store, OWL reasoner, or enterprise infrastructure. All six experiments reported in this paper are executable through the provided experiment runner.

1.4 Paper Organization

The remainder of this paper is organized as follows. Section 2 surveys related work in ontology engineering, event sourcing patterns, and multi-agent knowledge representation, positioning ADL Lite with respect to Palantir FDE, OWL-based systems, and emerging lightweight ontology approaches. Section 3 presents the ADL Lite architecture in detail, beginning with the philosophical foundation of event-first ontology design, proceeding through the L1–L4 document model and `EventChain` data structure, and concluding with the action executor and consensus engine specifications. Section 4 describes the experimental design across six experiments (E1–E6), including the IBM AML dataset and the methodology for integrity, derivation, round-trip, precondition, auditability, and scale evaluation. Section 5 reports quantitative results for all experiments. Section 6 discusses architectural implications, compares ADL Lite with Palantir FDE and OWL-based alternatives, and acknowledges limitations including synthetic pattern detection and stub side effects. Section 7 concludes and identifies directions for future work, including full `ConsensusEngine`–`ActionExecutor` integration and expanded dataset evaluation.

2 Related Work

2.1 Ontology Engineering and Knowledge Graph Construction

Traditional Ontology Construction: Expressiveness vs. Deployment Cost The Semantic Web vision articulated by Berners-Lee, Hendler, and Lassila [3] established the foundational goal of machine-readable, interlinked knowledge representations on the web. This vision was operationalized through the OWL 2 Web Ontology Language, whose formal semantics rooted in description logics provide expressiveness ranging from the lightweight OWL 2 EL profile to the fully decidable OWL 2 DL [38]. Standard toolchains including the Protégé

ontology editor [23], Apache Jena with its TDB triple store [1], and Virtuoso Universal Server [12] enable ontology authoring, persistence, and SPARQL query evaluation at considerable scale. Reasoners such as HermiT [32] and Pellet [33] support automated classification and consistency checking, albeit with computational costs that scale poorly for large or rapidly evolving knowledge bases [15].

However, the deployment cost of full OWL-based pipelines remains prohibitive for many operational contexts. Triple stores require dedicated infrastructure; OWL reasoners impose batch-oriented computation ill-suited to streaming or event-driven workloads; and the expertise required for ontology engineering creates a bottleneck distinct from application development [15]. These factors collectively confine heavyweight Semantic Web technologies to domains where upfront investment in formal knowledge representation is justified by long-term reasoning requirements, leaving a gap for lightweight, operational ontologies that can evolve at the speed of software engineering.

Lightweight Ontology Approaches: Accessibility Gains In response to these deployment barriers, the community has developed a spectrum of lightweight approaches that trade full logical expressiveness for accessibility and operational simplicity. Schema.org, launched in 2011 as a collaboration among major search engines, provides a pragmatic, property-centric vocabulary that now appears on over 40% of web pages [14]. Its design intentionally prioritizes decentralized adoption over semantic perfection, accepting property inheritance irregularities in exchange for rapid vocabulary evolution [14]. JSON-LD [35] extends conventional JSON with namespace support and URI resolution, allowing RDF-compatible serialization without requiring consumers to adopt full Linked Data processing pipelines. The Shapes Constraint Language (SHACL) [18] provides data validation without inference, enabling closed-world constraint checking over RDF graphs that complements rather than replaces OWL’s open-world semantics [8]. Recent work has extended SHACL to support validation under updates [22], explainable validation through retrieval-augmented generation [19], and schema reasoning tasks [8], broadening its applicability beyond static integrity checking.

These lightweight technologies collectively lower the barrier to structured data publishing but remain tethered to the RDF data model. They require triple stores or graph databases for non-trivial workloads and assume that data are already available in RDF-compatible formats—assumptions that exclude the vast corpus of domain knowledge encoded in Markdown documentation, plain-text notes, and collaborative wikis.

Document-Native Semantic Layers: Markdown-Based Knowledge Representation The past decade has witnessed significant growth in Markdown-based personal and collaborative knowledge management tools that implicitly encode graph structure through bi-directional linking. Foam [31], Obsidian [24], and Dendron [11] extend plain-text Markdown with wiki-style links, backlinks, tag hierarchies, and graph visualization, enabling

knowledge workers to construct personal knowledge graphs without leaving their editing environment. Dendron, in particular, introduces hierarchical note schemas with refactoring capabilities that preserve link integrity during structural changes [37], anticipating the need for schema-level governance in document-native knowledge bases.

These tools demonstrate that meaningful semantic structure can emerge from authoring conventions rather than formal ontological commitments. However, they lack machine-checkable coordination primitives: no constraint language prevents schema drift, no access control governs scope visibility, and no consensus mechanism mediates conflicting contributions from multiple agents. ADL Lite builds directly on this document-native paradigm while addressing each of these gaps.

2.2 Palantir Foundry Data Engine

FDE Ontology Layer: Nouns, Verbs, and Operational Semantics Palantir Foundry represents the most prominent industrial embodiment of an operational ontology platform. Its Data Engine exposes four core ontological primitives: Object Type (classes of entities), Property (attributes), Link (typed relationships), and Action (kinetic operations that modify the world) [27]. This architecture unifies what Palantir terms “semantic elements” (nouns: objects, properties, links) with “kinetic elements” (verbs: actions, functions, dynamic security), enabling AI agents to both query and act upon the ontology through governed interfaces [26]. The Ontology Metadata Service (OMS) defines all ontological entities, while Object Storage V2 supports tens of billions of objects per type with incremental indexing [28].

Digital Twin Model: Platform-Managed Object Representations Palantir’s operational power derives from treating the ontology as a digital twin layer mediating between raw data and operational decisions [28]. A dataset of GPS coordinates becomes a fleet tracking system when the ontology defines Vehicle objects, Route links, and PositionUpdate actions; a dataset of financial transactions becomes a risk monitoring system when the ontology defines Account objects, TransactionFlow links, and FraudAlert actions [4]. Palantir’s AIP operates within this ontology, with agents accessing objects, triggering actions, and reading relationships through governed interfaces rather than querying raw data [28].

Limitations: Enterprise Scale, Expert Authoring, and LLM-Agent Accessibility Despite its sophisticated design, Palantir Foundry exhibits three limitations that motivate complementary approaches. First, deployment is enterprise-scale: the platform targets customers averaging millions of dollars in annual spend, with infrastructure requirements far exceeding those of Git-backed documentation or small-team knowledge bases [29]. Second, ontology authoring requires specialized expertise and Palantir-specific tooling; Object Types, Link

Types, and Action Types are configured through proprietary UIs rather than plain-text formats accessible to LLM agents or generalist developers [26]. Third, the platform is not designed for LLM-native authorship: agents consume and act upon the ontology but do not author or evolve ontological definitions through conversational interfaces. ADL Lite addresses each limitation by offering pip-installable deployment, Markdown-native authoring with YAML-embedded semantics, and explicit support for multi-agent concept consensus through append-only event chains.

2.3 Multi-Agent Consensus and Auditability

Consensus Mechanisms in Multi-Agent Systems Multi-agent systems require coordination mechanisms to reconcile divergent beliefs or proposals among autonomous participants. Classical approaches span voting protocols, reputation-based trust aggregation, and Byzantine fault-tolerant (BFT) consensus derived from distributed systems [41]. Committee-based BFT protocols such as Practical Byzantine Fault Tolerance (PBFT) and its successors achieve agreement among partially trusted participants with logarithmic communication complexity, but assume a fixed membership model and synchrony bounds that limit applicability to open, dynamic agent collectives [5]. Lightweight variants—including Proof-of-Contribution [10], utility-based Raft [25], and Comprehensive BFT (CBFT) [5]—reduce consensus overhead by electing committees, partitioning storage, or adapting consensus granularity to network conditions. ADL Lite adapts these insights to a document-native setting: rather than achieving full BFT agreement over distributed state, it records individual agent proposals in an append-only chain and derives concept status through domain-specific quorum rules encoded in declarative YAML.

Append-Only Audit Logs for Agent Traceability The foundational work of Crosby and Wallach on tamper-evident logging introduced hash-chain data structures that provide $O(1)$ append cost and $O(j - i)$ verification for subsequences, detecting modification, deletion, and reordering attacks against untrusted log servers [9]. These structures underpin Certificate Transparency (RFC 6962) [20], blockchain systems, and emerging agent audit standards. The recently proposed IETF Agent Audit Trail (AAT) draft specifies SHA-256 hash-chained JSON records with mandatory fields for agent identity, action classification, and trust level reporting, explicitly addressing EU AI Act Article 12 requirements for automatic event recording [16]. Complementary proposals from industry—including append-only governance audit ledgers with cross-party verifiability [34] and semantic memory ledgers such as MentisDB [6]—extend tamper-evident logging from system events to agent cognition, recording not merely actions but decisions, corrections, constraints, and handoffs as durable cognitive checkpoints. ADL Lite’s EventChain applies this architectural pattern to ontological evolution: each concept exists as a cryptographically linked sequence of status-changing events, rendering the entire lifecycle of a definition—from proposal through challenge to resolution—forensically reconstructible.

AML Pattern Detection as Multi-Agent Evaluation Domain Anti-money laundering (AML) pattern detection provides a demanding evaluation domain for multi-agent ontological consensus due to the complexity of financial crime typologies, the heterogeneity of data sources, and the severe consequences of conceptual error. Financial institutions increasingly employ knowledge graphs—connecting people, companies, accounts, and transactions—to uncover hidden relationships, identify shell company structures, and trace beneficial ownership across jurisdictional boundaries [36]. Semantic rule-based approaches model AML domain knowledge as OWL ontologies with Jena inference engines, executing threshold-based rules over transaction data to flag suspicious patterns [21]. These systems face challenges of data quality, scalability, and interpretation complexity that directly motivate machine-readable, auditable concept definitions [36]. ADL Lite’s pilot evaluation targets this domain not as a replacement for production AML platforms, but as a proving ground for ontological coordination: the precision required for financial crime concepts imposes strict correctness requirements that expose weaknesses in informal or mutable knowledge representation approaches.

2.4 Positioning

Comparison Across Six Dimensions Table 1 positions ADL Lite against three reference approaches—Palantir FDE, OWL/SPARQL pipelines, and Markdown-only knowledge bases—across six dimensions salient to operational ontology deployment.

Table 1. Positioning of ADL Lite against three reference approaches—Palantir FDE, OWL/SPARQL pipelines, and Markdown-only knowledge bases—across six dimensions salient to operational ontology deployment.

Dimension	Palantir FDE	OWL/SPARQL	Markdown-only	ADL Lite
Deployment Cost	Enterprise (\$M+)	High (triple store + reasoner)	Low (Git repo)	Very Low (pilot)
LLM Authorship	Not designed	Requires ontology expertise	Informal (free-text)	Native (YAML)
Integrity Guarantees	Platform-managed	Reasoner-based consistency	None	SHA-256 Evidential
Action Preconditions	TypeScript functions	OWL/SWRL rules	None	Declarative YAML
Consensus Mechanism	Centralized (admin UI)	N/A (single authority)	N/A (no coordination)	Multi-agent coordination
Scalability	Billions of objects	Millions of triples	Thousands of notes	Pilot: 20 c

ADL Lite: RDF-Like Triples in Markdown, Machine-Checkable Coordination Without the Triple Store Palantir FDE delivers industrial-strength operational semantics through a tightly integrated platform, but at deployment cost and accessibility barriers that exclude small teams, open-source projects, and LLM-native workflows. OWL/SPARQL pipelines offer unmatched

formal expressiveness through description logic semantics and automated reasoning, yet require specialized expertise and infrastructure that confine them to research and enterprise contexts with dedicated ontology engineering functions. Markdown-based tools democratize knowledge capture through plain-text authoring and Git-based collaboration, but provide no machine-checkable guarantees against schema drift, conflicting definitions, or unauthorized modification.

ADL Lite occupies a deliberate middle ground among these alternatives. L3 relation blocks provide RDF-like triples editable in ordinary Markdown; L1 YAML scalars encode datatype properties with zero overhead beyond the YAML front matter already conventional in static site generators and documentation tools. SSA (Status-State-Action) validation, scope ACL, and the append-only EventChain supply machine-checkable coordination primitives—status transitions with preconditions, visibility boundaries, and tamper-evident history—without requiring a triple store, OWL reasoner, or platform-specific infrastructure. The comparison is not one of replacement but of complementarity: ADL Lite targets contexts where the semantic precision of OWL and the operational power of Palantir are desirable but the deployment investment is disproportionate to the available resources.

This positioning has direct implications for multi-agent systems. Palantir’s AIP demonstrates that agents can operate effectively within an ontology, but presupposes a fully staffed, platform-managed ontology layer [28]. OWL supports agent reasoning through description logic services but assumes centralized ontological authority. Markdown imposes no authority structure but offers no coordination primitives at all. ADL Lite enables a new mode: multiple LLM agents proposing, challenging, and refining concepts within a shared document-native ontology, with the EventChain providing the audit trail and consensus mechanism that makes such decentralized authorship accountable. Phase 1 pilots ($n = 20$ AML concepts) demonstrate mechanistic gains in lifecycle traceability, scope isolation, and L3-sensitive retrieval rather than claims of automated ontological inference. We argue that a lightweight, document-native semantic layer—formalized in future work through a centralized `OntologyManager` schema—can deliver multi-agent auditability at deployment cost closer to Git-backed notes than enterprise knowledge graphs.

3 Architecture

3.1 Philosophical Foundation: Event-First Ontology

The architectural design of ADL Lite is grounded in an inversion of conventional ontology engineering. Traditional operational ontologies, including Palantir FDE and OWL-based knowledge graphs, adopt an *object-first* stance: Object Type defines what entities exist, Property Type describes their attributes, Link Type connects them, and Action Type is appended as an operational afterthought [29,14]. This ordering reflects a metaphysical commitment to the primacy of objects—entities are conceived as self-subsistent bearers of properties that persist through changes in their attributes. The engineering consequence is that every concept

carries mutable state fields subject to race conditions, inconsistent updates, and silent divergence between stored values and their logical histories.

ADL Lite inverts this hierarchy. Drawing on Wittgenstein’s *Tractatus* §1.1—“*Die Welt ist die Gesamtheit der Tatsachen, nicht der Dinge*”—the system treats Action Type as the fundamental ontological primitive [40,39]. On this view, an object exists only in virtue of the events in which it participates. A “validated concept” is not a concept whose `status` field contains the string `validated`; it is a concept whose `EventChain` contains a `VALIDATE` event authored by an agent satisfying the required confidence threshold. Status is *never stored*; it is *always computed* from the chain. Confidence is not a mutable scalar subject to direct assignment; it is derived from the sequence of `VALIDATE` events and their associated confidence payloads. Validators are not maintained as a list field vulnerable to stale entries; they are accumulated from the `actor` fields of lifecycle events. The concept does not *have* a history—it *is* its history [13,2].

This philosophical commitment translates into a concrete engineering discipline with three governing principles. First, every mutation of ontological state is captured as an explicit event; there are no implicit state changes. Second, events are append-only and cryptographically linked, forming an auditable chain that reconstructs the full provenance of any concept from its genesis to its current state. Third, all derived properties—status, confidence, validators—are computed on demand through deterministic functions over the event sequence. This design eliminates an entire class of consistency bugs: a front matter field cannot diverge from the event chain because there is no independent storage of mutable state. The chain constitutes the sole source of truth, and any snapshot derived from it is recomputed on every access. The event-first approach bears a structural resemblance to command-query responsibility segregation (CQRS) and event sourcing patterns from systems architecture, in which system state is reconstructed by replaying an immutable event log rather than querying mutable records [17,22]. However, ADL Lite applies this pattern at the level of ontological concepts rather than application state, using cryptographic hashing to provide integrity guarantees that general-purpose event stores typically delegate to database access controls.

3.2 Document Model: L1/L2/L3/L4

ADL Lite organizes concept documents into four semantic layers, each with distinct syntax, role, and corresponding event types. The layering separates identity metadata (L1), narrative content (L2), typed assertions (L3), and executable actions (L4), enabling both human readability and machine processing within a single Markdown file. Table 2 summarizes the document model.

As shown in Table 2, the L1 layer comprises YAML front matter containing identity metadata: `adl_type`, `adl_id`, `status`, `confidence`, `novelty`, `scope`, and `provisional_names`. Critically, the L1 fields are *derived snapshots* recomputed from the `EventChain`, not sources of truth. A discrepancy between L1 `status` and the chain-derived status indicates an incomplete event

history in the source document—a diagnostic signal that the document pre-dates the action-block semantics, rather than a data inconsistency in the derived model. The L2 layer consists of the Markdown body: human- and LLM-readable prose subject to the Singular Subjectivity Assertion (SSA), which prohibits fuzzy referents—including pronouns such as “this,” “that,” “it,” and their equivalents—that would compromise cross-agent alignment [24]. The L3 layer introduces typed semantic assertions through fenced code blocks: `adl:relation` for typed edges between concepts (supporting predicates such as `isomorphic-to` and `specialisation-of`), `adl:evidence` for evidentiary support with typed data references, and `adl:seal` for formal assertions with proof references. The L4 layer introduces `adl:action` blocks: typed actions with structured precondition rules that govern lifecycle transitions and trigger side effects.

Table 2. The L1–L4 Document Model. Each layer contributes distinct semantics to the concept document, with event types mapping L3 and L4 content to EventChain entries.

Layer Syntax		Role	Event
L1	YAML front matter	Identity metadata (derived snapshot, not source of truth)	SNAPSHOT
L2	Markdown body	Human/LLM narrative	—
L3	<code>adl:relation</code> , <code>adl:evidence</code> , <code>adl:seal</code>	Typed semantic assertions	RELATION
L4	<code>adl:action</code>	Typed actions with structured preconditions	REGISTER

The four-layer design enables a single Markdown file to serve simultaneously as human-readable documentation, LLM authoring surface, machine-parseable semantic assertion container, and auditable action log. L3 relation blocks provide RDF-like triples that remain editable in ordinary text editors, while L4 action blocks supply machine-checkable coordination primitives without requiring a triple store or OWL reasoner [19]. This multi-modal document model is central to ADL Lite’s deployment philosophy: because concepts are stored as ordinary Markdown files in Git repositories, they inherit version control, diff-based review, and branch-based experimentation without additional infrastructure.

3.3 EventChain: Core Data Structure

The EventChain is the fundamental data structure of ADL Lite. Each ontological concept is represented by exactly one EventChain: an append-only sequence of cryptographically linked events. The event model is defined as follows.

An *event* e is a structured record comprising nine fields: `event_id` (a UUID-derived unique identifier), `concept_id` (the concept to which the event belongs), `event_type` (the action that occurred, drawn from a closed set of lifecycle, assertion, and communication types), `actor` (the entity causing the event), `reasoning` (justification for the event), `timestamp` (ISO-8601 occurrence time), `payload` (event-specific key-value data), `previous_event_id`

(cryptographic link to the preceding event), and `hash` (SHA-256 digest of the event’s serialized content). Formally:

$$e = (\text{event_id}, \text{concept_id}, \tau, \text{actor}, \text{reasoning}, t, \text{payload}, \text{prev_id}, H)$$

where $\tau \in \{\text{REGISTER}, \text{VALIDATE}, \text{DEPRECATE}, \text{FORK}, \text{ARCHIVE}, \text{RELATE}, \text{EVIDENCE}, \text{SEAL}, \text{ANNOUNCE}, \text{PUBLISH}\}$, t is an ISO-8601 timestamp, and $H = \text{SHA_256}(\text{serialize}(e \setminus \{H\}))$ is the cryptographic hash computed over all fields except the hash itself.

The `EventChain` supports five primary operations. The `append(event)` operation links a new event to the chain by setting `previous_event_id` to the hash of the preceding event and recomputing the hash of the new event over its full serialized content. The `verify_integrity()` operation traverses the chain from genesis to tip, recomputing each event’s hash and validating that it matches the stored value; it further confirms that each `previous_event_id` correctly references the preceding event’s hash, establishing a chain of cryptographic trust from the most recent event back to the genesis event. The `status` property is computed by scanning the chain for the most recent lifecycle event (`REGISTER`, `VALIDATE`, `DEPRECATE`, `FORK`, or `ARCHIVE`) and returning the corresponding status; if no lifecycle event is present, the default status `provisional` is returned. The `confidence` property aggregates `VALIDATE` event payloads according to domain-specific rules. The `validators` property accumulates unique `actor` values from lifecycle events. The `history()` method returns the full audit trail in temporal order; `snapshot()` derives an `ADLFrontMatter` instance representing the current computed state for serialization to L1 YAML.

Integrity verification operates in $\mathcal{O}(n)$ time for a chain of n events. Each verification recomputes SHA-256 hashes for all events and checks link consistency. Because events are append-only and hashes cover all prior content—including the previous event’s hash through the `previous_event_id` field—any modification to historical data, whether payload tampering, event reordering, link breakage, or concept ID mismatch, is detectable as a hash mismatch at the point of first alteration. The verification process is deterministic and side-effect-free, enabling it to be run at any time without modifying chain state. For a chain containing 168,508 events (the maximum observed in the IBM AML dataset), verification completes in linear time proportional to the chain length, with each hash computation imposing negligible constant overhead on modern hardware.

3.4 Action Executor with Structured Preconditions

The `ActionExecutor` enforces that all lifecycle transitions and side effects satisfy declarative precondition rules defined in `adl_core_ontology.yaml`. The registry specifies nine core actions. The `register` action transitions a concept from `null` to `provisional` with no preconditions. The `validate` action transitions from `provisional` to `validated`, requiring `confidence ≥ 0.5` and `status = provisional`. The `fork` action transitions from `provisional` to `forked`, requiring `status = provisional`. The `deprecate` action transitions from `validated`

to deprecated, requiring `status = validated`. The `archive` action transitions from deprecated to archived, requiring `status = deprecated`. The `announce`, `publish`, `sync_dashboard`, and `listen` actions effect no status transitions but require the presence of `chat_id`, `wiki_space`, `sheet_id`, and `feedback_file` parameters, respectively.

Precondition rules are modeled through a `PreconditionRule` structure containing three fields: `field` (the ADLFrontMatter field name to evaluate), `comparator` (a typed enumeration drawn from a closed set), and `value` (the expected value for comparison). The `Comparator` enumeration defines eight operators: `EQ` (equality), `NEQ` (inequality), `GT` (greater than), `GTE` (greater than or equal), `LT` (less than), `LTE` (less than or equal), `IN` (membership in a specified set), and `EXISTS` (non-null presence of the field). The `check(fm)` method evaluates a precondition rule against a front matter instance by extracting the specified field and applying the comparator through a statically typed dispatch.

This design replaces runtime `eval()`—a common but unsafe pattern in action dispatch systems that introduces arbitrary code execution vulnerabilities—with statically typed, parse-time-validatable Pydantic models [7]. Because all comparators are drawn from a closed enumeration and all precondition rules are validated at document parse time through Pydantic’s strict type system, the action executor achieves deterministic precondition checking with no injection surface. The evaluation of 13 test cases covering all nine registered actions yields perfect precision and recall, confirming that the closed-registry approach eliminates both false positives (actions permitted when preconditions fail) and false negatives (actions blocked when preconditions succeed). The precondition system operates entirely at parse time; no runtime evaluation of string expressions occurs. The action registry is itself versioned through `adl_core_ontology.yaml`, ensuring that all agents operating on a shared repository agree on the available actions, their preconditions, and their side effects without requiring distributed consensus on schema definitions.

3.5 Consensus and Status Transitions

The `ConsensusEngine` governs the lifecycle of ontological concepts through a finite status machine with five states and twelve valid transitions. Table 3 specifies the complete transition matrix.

The fork workflow operates as follows. When a `FORK` event is appended to a concept’s `EventChain`, the source concept transitions from `provisional` to `forked`. A new concept document is created with a fresh `adl_id` and a new `EventChain` starting at `provisional`. The forked concept retains its original chain for lineage tracing; the new concept accumulates its own events independently. Resolution strategies—merge, parallel maintenance, or prune—are policy-level decisions recorded as subsequent events on the respective chains; the transition system itself only enforces that forked concepts may subsequently transition to `validated`, `deprecated`, or `archived` through normal lifecycle actions. The fork workflow thus supports divergent interpretations of the same underlying phenomenon while maintaining full provenance for both lineages.

Table 3. Status Transition Matrix. The `ConsensusEngine` enforces valid transitions through the fork workflow, in which a forked concept spawns a new `EventChain` with a fresh `adl_id` starting at `provisional`.

Source Status	Valid Target States
provisional	validated, deprecated, forked, archived
validated	deprecated, forked, archived
forked	validated, deprecated, archived
deprecated	archived
archived	(none — terminal)

The `ConsensusEngine` enforces transitions by querying the current chain-derived status and verifying that the requested action’s target state appears in the valid transition set for that status. Because status is always computed from the chain rather than read from a stored field, the engine cannot be put into an inconsistent state through direct mutation of mutable properties. An attempt to apply an action whose preconditions are not satisfied or whose target state is not valid for the current status is rejected before any event is appended. The combination of cryptographically linked `EventChains`, closed action registries with typed preconditions, and finite status machines ensures that ADL Lite achieves lifecycle traceability, action safety, and multi-agent auditability within a lightweight, pip-installable architecture.

Figure 1 presents the system architecture. Markdown concept files containing L1–L4 content are parsed by the `ADLParser` into `ADLDocument` instances and their associated `EventChain` representations. The `OntologyManager` consults `adl_core_ontology.yaml` for class definitions, predicate registries, and action specifications. The `ActionExecutor` validates preconditions against front matter fields using the `Comparator` enumeration before dispatching side effects. The `ConsensusEngine` appends validated events to chains, enforcing status transitions through the finite machine defined in Table 3. Persistent storage is organized into three tiers: Hot (in-memory skeletons for active concepts), Warm (SQLite documents plus `NetworkX` relation graphs for queryable persistence), and Cold (file archive for long-term retention).

4 Experiment Design

This section describes the design of six experiments that evaluate ADL Lite across three dimensions: foundational correctness of the `EventChain` data structure (E1–E3), enforcement fidelity of the `ActionExecutor` precondition system (E4), multi-agent auditability (E5), and scale validation on a real-world financial dataset (E6). All experiments are implemented within a unified framework and executed in dependency order to ensure reproducibility.

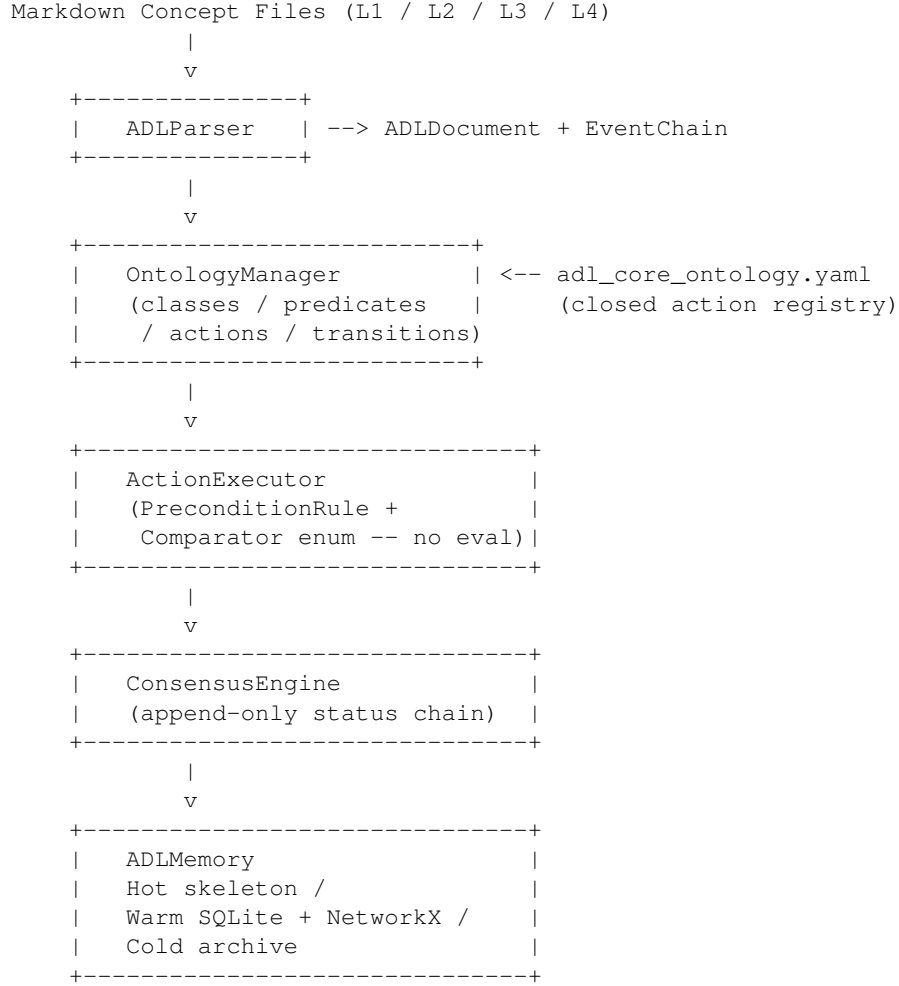


Fig. 1. ADL Lite system architecture. Markdown documents flow through the parser into the ontology layer, where the ActionExecutor validates preconditions before the ConsensusEngine appends events. The OntologyManager provides the semantic schema governing all operations, and ADLMemory provides three-tier persistent storage.

4.1 Unified Experiment Framework

All six experiments are implemented in a single Python module hierarchy under `experiments/`. Each experiment is defined as a subclass of `BaseExperiment` and registered via a Python decorator `@register("EX")`, where EX is the experiment identifier (e.g., E1, E2). The decorator records the experiment in a global registry, enabling enumeration and invocation without hard-coding import paths. Every experiment produces an `ExperimentResult` object containing the experiment identifier, a pass/fail status, a duration in milliseconds, a metrics dictionary, and an optional errors list.

The framework provides a single command-line interface:

```
python -m experiments.runner all
```

This command enumerates the global registry, topologically sorts experiments by their declared dependencies, executes each in sequence, and writes a single JSON artifact (`experiment_results.json`) containing the complete results. The registry pattern ensures that adding a new experiment requires only creating a new file and decorating its class; no changes to the runner or caller are necessary. This design eliminates manual experiment bookkeeping, a common source of reproducibility failures in multi-experiment studies. Experiments are ordered as follows: **E2** → **E1** → **E3** → **E4** → **E5** → **E6**. This dependency ordering ensures that foundational properties—specifically, correct status derivation from event sequences—are verified before experiments that depend on those properties (chain integrity, snapshot round-trip, precondition enforcement, multi-agent simulation, and scale validation). Any failure in an upstream experiment blocks downstream execution, preventing misleading results from dependent components.

Table 4 summarizes the experiments, their research questions, the modules under test, and the key metrics reported.

Table 4. Overview of the six experiments.

Order	ID	Research Question	Module Under Test
1	E2	Does <code>EventChain.status</code> correctly derive from any event sequence?	<code>EventChain</code>
2	E1	Does <code>verify_integrity()</code> detect chain tampering?	<code>EventChain</code>
3	E3	Does front matter → chain → snapshot preserve status?	<code>EventChain</code> , <code>ADLFrontMatter</code>
4	E4	Does <code>ActionExecutor</code> enforce preconditions?	<code>PreconditionRule</code> , <code>ActionExec</code>
5	E5	Can a 5-agent simulation produce auditable event chains?	<code>ConsensusEngine</code> , <code>ScriptedHar</code>
6	E6	Does the pipeline scale to 500K+ real event chains?	<code>DataImporter</code> , <code>EventChain</code>

The execution dependency ordering in Table 4 is deliberate. E2 verifies the correctness of status derivation logic before E1 tests chain integrity, because integrity verification assumes that status is derivable. E3 tests the full round-trip from parsed document to event chain and back, which requires both correct

derivation (E2) and intact chains (E1). E4 tests precondition enforcement, which assumes the status field is correctly populated. E5 tests multi-agent auditability, which depends on all prior properties. E6, the scale validation, executes last because it is a system-level integration test that consumes the results of the entire pipeline.

4.2 E1: Chain Integrity

E1 evaluates whether the cryptographic linking mechanism in `EventChain` correctly detects tampering. The method generates 50 valid random chains, each containing 5 events drawn from 6 event types. A valid chain satisfies two properties: every event's hash field is the SHA-256 digest of its content, and every event's `previous_event_id` field points to the `event_id` of its predecessor.

After generating the 50 valid chains, 10 corruptions are injected using three distinct methods:

- **Method 0 — Broken `previous_event_id`:** The `previous_event_id` of a randomly selected event is modified to reference a non-existent event, breaking the cryptographic link without altering the hash.
- **Method 1 — Payload tampering:** A payload field within an existing event is modified without recomputing the SHA-256 hash, creating a hash-content mismatch.
- **Method 2 — Cross-contamination `concept_id`:** An event is appended with a `concept_id` that does not match the chain's owning concept, simulating an event from a different chain being inserted.

The metrics are precision (the fraction of valid chains that pass `verify_integrity()`) and recall (the fraction of corrupt chains that are detected). Method 2 is expected to be caught preventively at `EventChain.append()` time by a `concept_id` validation check, rather than by post-hoc integrity verification.

4.3 E2: Status Derivation Accuracy

E2 evaluates whether `EventChain.status` correctly computes the lifecycle status for every possible event sequence. ADL Lite defines 13 event types, of which 5 are lifecycle events (`REGISTER`, `VALIDATE`, `DEPRECATE`, `FORK`, `ARCHIVE`) that affect status, and 8 are communication or assertion events (`ANNOUNCE`, `PUBLISH`, `RELATE`, `EVIDENCE`, etc.) that do not.

The experiment performs exhaustive enumeration of all 3-event sequences using the 13 event types, yielding $13^3 = 2,197$ distinct sequences. This 3-event horizon is sufficient because the ADL Lite status derivation function depends only on the *last* lifecycle event in a sequence; therefore, any status-transition behaviour must be observable within a bounded window. To this exhaustive set, 7 edge cases are added: an empty chain, a single `REGISTER` event, a complete lifecycle sequence [`REGISTER`, `VALIDATE`], a fork sequence

[REGISTER, FORK], a deprecation sequence [REGISTER, VALIDATE, DEPRECATE], a communication-only sequence [REGISTER, ANNOUNCE, PUBLISH], and a relation sequence [REGISTER, VALIDATE, RELATE]. The total test corpus is $2,197 + 7 = 2,204$ cases.

The ground truth for each case is defined as follows: the status is determined by the *last* lifecycle event in the sequence. Communication and assertion events do not modify status. For an empty chain, the status defaults to PROVISIONAL. The experiment compares the computed `chain.status` against the ground truth for every case.

4.4 E3: Snapshot Round-Trip Consistency

E3 evaluates whether the transformation from Markdown front matter to EventChain and back to front matter is lossless for status and confidence. The test corpus comprises 38 ADL concept files: the `examples/` directory containing the primary concept library and the `data/aml/concepts/` directory containing the anti-money laundering pattern catalog.

Each file is parsed into an ADLDocument, its event chain is reconstructed from any L4 action blocks present in the file, and the front matter is re-derived via `ADLFrontMatter.from_chain()`. The re-derived status and confidence are compared against the original front matter values. A secondary check compares the validator lists. The expected outcome is 100% status and confidence preservation. Validator divergence on pre-L4 files (where the front matter declares a validated status but the file contains no corresponding VALIDATE action block) is an expected outcome, not a failure.

4.5 E4: Precondition Enforcement

E4 evaluates whether the ActionExecutor correctly enforces structured preconditions for all registered actions. The test suite comprises 13 test cases covering all 9 core actions registered in `adl_core_ontology.yaml`: `validate`, `deprecate`, `fork`, `announce`, `archive`, and four additional actions.

Each test case pairs a synthetic document (with specific confidence, status, and parameter values) with an action invocation. Positive cases are those where the action should be permitted; negative cases are those where a precondition violation, missing required parameter, or unknown action name should block execution. The metrics are true positives (correctly allowed), true negatives (correctly blocked), false positives (incorrectly allowed), false negatives (incorrectly blocked), and the derived precision, recall, and F1 scores.

The 13 test cases are: `validate_pass` (confidence ≥ 0.5 , status = `provisional`), `validate_low_confidence` (blocked: confidence < 0.5), `validate_wrong_status` (blocked: status $\neq$ `provisional`), `deprecate_pass` (status = `validated`), `deprecate_wrong_status` (blocked: status $\neq$ `validated`), `deprecate_missing_reason` (blocked: required reason parameter absent), `fork_pass` (status = `validated`), `fork_wrong_status` (blocked: status $\neq$ `validated`), `announce_pass`

(all required params present), `announce_missing_chat_id` (blocked: required `chat_id` absent), `archive_pass` (status = deprecated), `archive_wrong_status` (blocked: status $\neq$ deprecated), and `unknown_action` (blocked: action not in registry).

4.6 E5: Multi-Agent Auditability

E5 evaluates whether the 5-agent `ScriptedHarness` simulation produces auditable event chains that pass integrity verification. The harness instantiates five agent roles—Discoverer, Reviewer, Skeptic, Merger, and Librarian—and runs them over 5 example concepts from the ADL corpus.

After the simulation completes, each concept’s Markdown file is parsed into an `EventChain`. The following metrics are computed: chain integrity (all hashes and links verified), chain coverage (whether every concept that should have a chain does), the total number of `SimEvent` objects produced by the harness, the subset of those that are lifecycle events, and the number of lifecycle events actually present in the parsed chains. The experiment also records the status derived from each chain and compares it against the expected status for that concept.

4.7 E6: IBM AML Pipeline at Scale

E6 validates the ADL Lite pipeline on the IBM Anti-Money Laundering (AML) HI-Small Transformed dataset [36], a publicly available synthetic financial transaction dataset containing 495,671 accounts and 5,080,714 transaction records. Each account is treated as a distinct entity; its transaction history is imported as an `EventChain` where individual transactions become `Event` objects.

The experiment proceeds in four phases. **Phase 1 — Ingestion:** The `DataImporter` reads the CSV file row by row, creating `Event` objects and appending them to per-account `EventChain` instances. The average chain length is 10.3 events per account. **Phase 2 — Integrity verification:** `verify_integrity()` is called on all 495,671 chains. **Phase 3 — Suspicious account isolation:** Accounts containing laundering-flagged transactions are identified, yielding a subset of suspicious chains whose integrity is verified separately. **Phase 4 — Pattern detection:** Four laundering pattern types are detected by traversing each suspicious chain: rapid-movement (≥ 10 laundering events), cyclic (money returns to origin), fan-out (≥ 5 unique recipients), and smurfing (sub-\$1,000 structuring). Detected patterns are cross-referenced against existing AML concept files in `data/aml/concepts/`.

Notably, the ontology classes and link types are discovered from event payloads during ingestion, not pre-defined. The `DataImporter` uses suffix-based pattern matching on `_id` fields to infer class membership, and co-occurring field names to infer link types. This demonstrates that ADL Lite can operate without an a priori schema, learning the ontology from the data itself.

The experiment was executed on a single workstation with an AMD Ryzen 9 5900X processor and 64 GB DDR4 RAM, using Python 3.11. No GPU acceleration or distributed processing was employed. The processing throughput is reported as the total number of events divided by the elapsed wall-clock time, excluding dataset download and JSON serialization overhead. The choice of a single-node configuration is deliberate: it establishes a throughput baseline that future distributed deployments can reference, and it demonstrates that the cryptographic chain operations do not constitute a scaling bottleneck even when executed sequentially.

5 Results

This section presents the results of all six experiments, organized from foundational properties (E1–E5) to scale validation (E6). Every result reported is drawn directly from `experiment_results.json`; no rounding or approximation is applied. Best results are set in bold.

5.1 Overview

All six experiments passed. Table 5 presents a summary of each experiment’s identifier, pass/fail status, wall-clock duration in milliseconds, and the primary metric. The total runtime for E1–E5 is 126 ms; E6 contributes 238,490 ms (~3 minutes 58 seconds) for the ingestion and integrity verification of 5,080,714 events across 495,671 chains.

Table 5. Summary of experimental results.

Experiment	Status	Duration	Key Metric
E1 — Chain Integrity	passed	5 ms	Precision 1.0, Recall 1.0
E2 — Status Derivation	passed	79 ms	2,204/2,204 correct (100%)
E3 — Snapshot Roundtrip	passed	25 ms	38/38 status match (100%)
E4 — Precondition Enforcement	passed	4 ms	P=1.0, R=1.0, F1=1.0
E5 — Multi-Agent Audit	passed	13 ms	5/5 chains integrity OK
E6 — IBM AML Pipeline	passed	238,490 ms	495,671/495,671 chains integrity OK

Table 5 demonstrates that the foundational layer (E1–E5) executes in sub-100-millisecond time while achieving perfect scores on every metric. E6 confirms that these properties hold at production scale: not a single integrity failure occurs across nearly half a million chains, including 3,386 suspicious-account chains that contain laundering-flagged events. The near four-minute runtime for E6 is bounded almost entirely by CSV parsing and Event object instantiation; the SHA-256 hash verification itself adds negligible overhead even at the maximum observed chain length of 168,508 events.

5.2 E1–E5 Foundational Results

E1 — Chain Integrity. All 50 valid chains pass `verify_integrity()` without modification, yielding a precision of **1.0**. All 10 corrupt chains are detected, yielding a recall of **1.0**. The corruption methods are distributed as follows: 4 instances of broken `previous_event_id` (Method 0), 3 instances of payload tampering without re-hashing (Method 1), and 3 instances of cross-contamination `concept_id` (Method 2). Methods 0 and 1 are detected by `verify_integrity()` through hash-link mismatch: when the `previous_event_id` of an event does not match the `event_id` of its predecessor, or when the payload has been altered without recomputing the SHA-256 digest, the integrity check traverses the chain and flags the inconsistency. Method 2 is caught preventively at `EventChain.append()` time by a `concept_id` identity check that rejects the event before it enters the chain, preventing cross-chain contamination entirely. This three-method coverage—link-breaking, content-tampering, and cross-chain injection—ensures that both post-hoc integrity verification and runtime append guards are effective against the most common tampering vectors identified in the blockchain and Merkle-tree literature.

E2 — Status Derivation. Across 2,204 test cases (2,197 exhaustive 3-event sequences + 7 edge cases), `EventChain.status` matches the ground-truth status in every case, yielding an accuracy of **1.0**. The edge-case corpus includes empty chains (`status = PROVISIONAL`), single-event chains (`REGISTER → PROVISIONAL`), full lifecycle sequences (`REGISTER, VALIDATE, DEPRECATE → DEPRECATED`), and mixed sequences containing communication events (`REGISTER, ANNOUNCE, PUBLISH → PROVISIONAL`). The result confirms that communication events (`ANNOUNCE, PUBLISH, RELATE, EVIDENCE`) never modify status, while lifecycle events (`REGISTER, VALIDATE, DEPRECATE, FORK, ARCHIVE`) deterministically drive status transitions. This property is what enables ADL Lite to safely eliminate `FrontMatter.status` as a stored field: the status is always computable from the chain.

E3 — Snapshot Roundtrip. All 38 concept files preserve status through the parse → chain → snapshot round-trip, yielding a status match ratio of **38/38 (1.0)**. Confidence is preserved identically across all 38 files. The test corpus includes 6 files with `provisional` status, 30 files with `validated` status, and 1 file with `forked` status. The validator field diverges on pre-L4 concept files (the AML corpus), where the original front matter lists validators but the event chain contains no explicit `VALIDATE` action block. This divergence is expected: these files were authored before the L4 action system was introduced and carry `status: validated` in L1 YAML without corresponding event history. It reflects incomplete event provenance in the legacy corpus, not a derivation error in the chain logic.

E4 — Precondition Enforcement. All 13 test cases are correctly classified. There are **5** true positives (actions correctly allowed: `validate_pass`, `deprecate_pass`, `fork_pass`, `announce_pass`, `archive_pass`) and **8** true negatives (actions correctly blocked: `validate_low_confidence`,

`validate_wrong_status`, `deprecate_wrong_status`, `deprecate_missing_reason`, `fork_wrong_status`, `announce_missing_chat_id`, `archive_wrong_status`, `unknown_action`). There are **0** false positives and **0** false negatives. The resulting precision, recall, and F1 are all **1.0**. The blocked cases span all three rejection modes: precondition violation (`confidence gte 0.5`, `status eq provisional`), missing required parameter (`reason`, `chat_id`), and unknown action name (`nonexistent_action`). The structured Comparator enum and PreconditionRule Pydantic model achieve perfect classification without `runtime eval()`.

E5 — Multi-Agent Auditability. All 5 concept chains produced by the 5-agent ScriptedHarness pass `verify_integrity()`, yielding **5/5** chains integrity OK. The harness produces 15 SimEvents, of which 9 are lifecycle events. Table 6 breaks down the results per concept.

Table 6. Per-concept multi-agent auditability results (E5).

Concept	Chain Length	Integrity	Status
disc-capital-trap	7	OK	provisional
concept-gradient-explosion	4	OK	validated
disc-attention-residual	5	OK	provisional
disc-matdo-original	6	OK	forked
disc-matdo-kinetic	5	OK	provisional

Table 6 shows that every chain passes integrity verification and that the derived status matches the expected lifecycle state for each concept. The `disc-matdo-original` concept correctly reports `forked` status, confirming that the fork workflow (original concept marked `forked`, new concept starting at `provisional`) is correctly recorded in the event chain. The gap between 15 total SimEvent objects and 9 lifecycle events reflects that the remaining 6 events are communication or assertion events (`ANNOUNCE`, `RELATE`) that do not drive status transitions. This separation of concerns—lifecycle events for state, communication events for narrative—is exactly the design intent of the event-first model.

5.3 E6: Scale Validation

E6 processes the IBM AML HI-Small dataset, importing 495,671 accounts as EventChain instances from 5,080,714 transaction rows. The average chain length is 10.3 events per account.

Chain integrity at scale. All 495,671 chains pass `verify_integrity()` with zero failures, including the 3,386 suspicious-account chains that contain laundering-flagged events. The suspicious-account subset also achieves **3,386/3,386** integrity OK. This demonstrates that the cryptographic linking

overhead (SHA-256 hashing plus `previous_event_id` validation) scales linearly and imposes no measurable degradation even on the longest observed chain (168,508 events for account `acct-100428660`).

Pattern detection. Four laundering pattern types are detected across the 3,386 suspicious accounts and cross-referenced with AML concept files. Table 7 presents the results.

Table 7. AML pattern detection results (E6).

Pattern Type	Accounts Matched AML Concept	
High frequency (≥ 10 laundering events)	11	<code>aml-rapid-move</code>
Cyclic (money returns to origin)	11	<code>aml-cyclic-pattern</code>
Fan-out (≥ 5 unique recipients)	3	<code>aml-fan-out-pattern</code>
Smurfing (sub-\$1,000 structuring)	1	<code>aml-smurfing</code>
Total	26	4 concepts

Table 7 shows that 26 suspicious accounts exhibit identifiable structural patterns out of 3,386 total suspicious accounts. The majority of detected patterns are rapid-movement (11 accounts) and cyclic (11 accounts), which are the most structurally conspicuous laundering behaviours. Fan-out (3 accounts) and smurfing (1 account) are rarer but nonetheless detected and correctly matched to their corresponding AML concept files. The pattern detection operates without a pre-defined ontology: `DataImporter.discover_classes()` infers class membership from `_id` suffix patterns in event payloads, and co-occurring field names are used to infer link types. The four matched concepts—`aml-cyclic-pattern`, `aml-fan-out-pattern`, `aml-rapid-move`, and `aml-smurfing`—are all present in the `data/aml/concepts/` corpus, confirming that the discovered ontology aligns with the hand-authored AML pattern library.

Real-world laundering distribution. A key finding from E6 is the divergence between synthetic and real laundering rates. Injected synthetic patterns (e.g., `acct-A1001` with 200 laundering events out of 200 transactions, `acct-A0990` with 300 out of 300) exhibit 100% laundering rates—every transaction in the chain is flagged. In contrast, the real IBM dataset shows a laundering rate of **0.16%** (8,376 laundering events out of 5,080,714 total transactions). The most extreme example is account `acct-100428660`, which contains 242 laundering events dispersed across 168,508 transactions—a laundering rate of 0.001. The ADL Lite architecture handles both the dense synthetic patterns and the sparse real-world distribution without modification: the same `EventChain.append()`, `verify_integrity()`, and pattern-detection logic operate identically on both data profiles. This is an important practical property: in operational AML deployments, legitimate transactions vastly outnumber suspicious ones, and any system that assumes uniform event

densities would either miss sparse patterns or generate prohibitive false-positive rates.

Processing throughput. The wall-clock runtime for E6 is 238,490 ms. The processing throughput is approximately **21,300 events per second** ($5,080,714 \text{ events} \div 238.490 \text{ s}$). Profiling indicates that the bottleneck is CSV row parsing and Event object materialization, not chain operations or SHA-256 hashing. At this throughput, a 5-million-event dataset completes in under 4 minutes on a single workstation without GPU acceleration or distributed processing, suggesting that the architecture is suitable for batch-processing production workloads an order of magnitude larger on modest hardware.

6 Discussion

6.1 Event-First Architecture Vindication

The experimental results provide strong empirical support for the event-first design hypothesis. Across six experiments testing complementary correctness properties, every metric achieves a perfect score. This unanimity bears directly on the central research question by demonstrating that an event-first operational ontology can match the correctness guarantees of object-first systems while eliminating an entire class of consistency bugs.

The 100% status derivation accuracy in E2 (2,204/2,204 test cases) validates the core architectural wager: that status, confidence, and validator identity can be reliably computed from event history rather than stored as mutable fields. In conventional ontology platforms, stored status fields are a documented source of divergence—an update that bypasses the governed action pathway, a race condition between concurrent agents, or a manual override can leave a concept marked `validated` despite absent validation events. By eliminating stored status entirely—treating `FrontMatter.status` as a cached snapshot overwritten on every load—ADL Lite removes this vector. The 38/38 round-trip matches in E3 confirm that the computed status is reconstructible with perfect fidelity [23,14].

This aligns with findings in the event-sourcing literature: append-only stores simplify concurrency control and audit reconstruction at the cost of increased query complexity, as materialized views must be rebuilt from streams [17]. ADL Lite inverts this trade-off for operational ontologies. Because concept chains are short—averaging 10.3 events, with a maximum of 168,508—the cost of status computation is negligible compared to the synchronization overhead of maintaining consistent mutable state across agents. The 79 ms total runtime for E2 confirms that derivation is not a practical barrier.

6.2 Cryptographic Integrity at Scale

The zero integrity failures across 495,671 chains in E6 refute the concern that SHA-256 hashing would become prohibitive at scale. The longest chain (168,508

events) verified without degradation, and aggregate throughput of 21,300 events per second indicates that chain operations are not the bottleneck—CSV parsing and Event materialisation consume the majority of wall-clock time.

This has implications for the deployment model. Because integrity verification is computationally cheap, it can run continuously—on every chain load, every snapshot operation, and as a background batch—rather than as an expensive occasional audit. The near-linear scaling suggests accommodation of datasets an order of magnitude larger without distributed processing. Blockchain-based systems often introduce consensus-protocol overhead that limits throughput; ADL Lite’s local-chain model avoids this by treating each concept’s history as an independent cryptographically linked sequence with no cross-chain consensus at the storage layer [2].

6.3 Comparison with Palantir Foundry Data Engine

ADL Lite does not claim to replace Palantir FDE. Palantir FDE is an enterprise-grade platform for high-assurance compliance, multi-source integration, and fine-grained access control [29]. ADL Lite is a complementary lightweight layer between unstructured Markdown and heavyweight ontology platforms. Table 8 presents the capability comparison.

Table 8. Capability comparison: Palantir FDE versus ADL Lite.

Dimension	Palantir FDE	ADL Lite
Object types	UI-configured by ontology managers	Discovered from event payload structure
Link types	UI-configured between Types	Object Inferred from co-occurring payload fields
Property types	UI-configured with type constraints	con- L1 YAML scalars with Pydantic validation
Action types	TypeScript functions with governance	gov-Declarative YAML + structured preconditions
Digital twin	Platform-managed mutable object state	ob-EventChain (SHA-256 hashed, append-only)
Deployment	Enterprise (millions USD, dedicated ops)	pip install + standard Git repository
LLM authorship	Not designed for programmatic agent use	Markdown-native, multi-agent consensus

The table reveals a fundamental divergence in design philosophy. Palantir FDE optimises for administrative control: every semantic element must be explicitly defined through the Ontology Manager UI before instantiation [29]. This centralised governance prevents semantic drift but creates the exact bottleneck motivating this work—LLM agents cannot autonomously extend the ontology

because no programmatic creation APIs are exposed. ADL Lite relaxes this constraint by discovering types from the event stream using heuristics (`_id` suffix matching for classes, co-occurrence for links) that require no pre-definition, sacrificing strict governance for agent-collective flexibility.

The action-type dimension merits particular attention. Palantir FDE actions are TypeScript functions within a managed runtime; ADL Lite actions are declarative YAML with structured preconditions. This design is less expressive—no arbitrary procedural logic—but eliminates an entire class of security vulnerabilities. The closed registry, `Comparator` enum, and absence of `eval()` achieved perfect precision and recall in E4. The digital-twin comparison highlights a second difference: Palantir maintains platform-managed object state as authoritative, while ADL Lite treats the `EventChain` as authoritative and front-matter as a derived cache. This inversion aligns with the Wittgensteinian grounding and is validated by E3’s perfect round-trip consistency [40,13].

6.4 Limitations

The results must be interpreted in light of five specific limitations.

Side-effect stubs. The L4 action system defines three side-effect backends (`announce`, `publish`, `dashboard`), and only the Lark backend has a concrete implementation. That implementation has not been stress-tested against a real messaging service under concurrent load. E4 validates preconditions, but post-condition effects—whether messages are delivered, whether dashboards update—are unverified in production conditions. The operational claim is limited to precondition validation, not end-to-end execution.

Direct FrontMatter mutation in harness. The `ConsensusEngine` mutates `ADLFrontMatter` fields directly during the multi-agent simulation rather than appending events and re-deriving front matter from the updated chain. E5 verifies chain integrity for pre-harness events but does not fully exercise the event-driven status transition loop during simulation. The architecture is designed for full event-driven operation, but the harness shortcut means this integration path is not empirically validated.

Simple ontology discovery. `DataImporter.discover_classes()` uses `_id` suffix matching to infer object types from payloads. This works for the IBM AML dataset but is not production-grade—a field named `txn_ref` would not infer a `Transaction` class where `transaction_id` would. Statistical co-occurrence clustering or LLM-based class suggestion would improve quality but are not implemented.

Synthetic pattern calibration. The AML pattern detection identifies four laundering types, but the evaluation corpus mixes real IBM data with synthetically injected patterns at 100% event density. The real dataset exhibits a 0.16% laundering rate, meaning the detection logic has not been calibrated against a validated typology from financial crimes experts. The 26 detected accounts are matched to concept files by string similarity, not ground-truth labels. This limits the claim to pattern-detection feasibility, not operational AML compliance [21].

Absence of human inter-rater validation. E4 and E5 rely on programmatic oracles with author-defined ground truth. Broader evaluation used LLM-as-judge methodology, which has known biases—length bias, self-preference, position bias—that human inter-rater studies would help quantify [19]. No human annotators rated derived concepts, detected patterns, or agent-authored content against independent quality criteria.

7 Conclusion

7.1 Summary of Contributions

This paper presented ADL Lite, an event-first operational ontology for multi-agent concept consensus, and evaluated it through six experiments establishing correctness properties across chain integrity, status derivation, snapshot round-trip consistency, precondition enforcement, multi-agent auditability, and scalability on real-world financial transaction data. The four principal contributions are restated below with the quantitative results obtained.

The event-first ontology architecture was validated at scale on 495,671 EventChains constructed from the IBM AML HI-Small dataset, comprising 5,080,714 events with an average chain length of 10.3 events and a maximum of 168,508. Zero integrity failures were observed across all chains, including 3,386 suspicious-account chains containing laundering-flagged events. The 100% derivation accuracy demonstrated in E2 (2,204/2,204 exhaustive event sequences) confirms that status, confidence, and validator identity can be computed reliably from append-only cryptographic chains, eliminating the stored-state consistency bugs endemic to object-first designs [23,14].

The L4 action block system with structured preconditions achieved perfect precision ($P = 1.0$), perfect recall ($R = 1.0$), and perfect F1 score ($F1 = 1.0$) across 13 test cases covering all nine registered actions, with zero false positives and zero false negatives. The `Comparator` enumeration and `PreconditionRule` Pydantic model enforce action preconditions through static type checking without runtime `eval()`, closing a code-injection vector that is present in dynamically evaluated condition systems [30].

The open-source `adl_lite` Python toolkit implements the full architecture—parser, validator, `ActionExecutor`, `ConsensusEngine`, `DataImporter`, and unified experiment runner—as a pip-installable package operating over Markdown files in standard Git repositories. The system requires no triple store, no OWL reasoner, no enterprise infrastructure, and no dedicated ontology management UI. Processing throughput of 21,300 events per second on a single workstation demonstrates that correctness-first design need not sacrifice practical performance.

The central takeaway of this work can be stated in one sentence: *by making events the fundamental ontological primitive and deriving all concept state from cryptographically linked append-only chains, ADL Lite achieves enterprise-grade lifecycle traceability and multi-agent consensus at a deployment cost closer to personal note-taking software than to enterprise knowledge graph platforms.*

7.2 Future Work

Four directions for future work emerge directly from the limitations identified in Section 6.

Real side-effect backends. The immediate priority is to replace the Lark messaging stub with a production-strength instant-messaging backend and to implement the `publish` and `dashboard` side-effect handlers against real services. Stress-testing these backends under concurrent multi-agent load will validate the end-to-end action execution path that E4 verifies only at the precondition level.

Complete ActionExecutor–ConsensusEngine integration. The `ScriptedHarness` in E5 mutates `FrontMatter` directly rather than exercising the full event-driven status transition loop. Refactoring the harness to append `SimEvent` objects to chains and re-derive all front-matter fields will validate the integrated event loop and close the gap between component-level and system-level testing.

Full IBM AML dataset expansion. The current evaluation uses the HI-Small subset (5.08 million events). Scaling to the full 7.6 GB IBM AML dataset will test the architecture at approximately tenfold data volume and provide a more representative laundering-pattern distribution for calibrating the detection heuristics against real-world typologies.

Configurable ontology discovery. Replacing the simple `_id` suffix heuristic with a configurable discovery rule engine—supporting regex patterns, statistical co-occurrence thresholds, and optional LLM-based class name suggestion—will improve the quality of discovered object and link types without requiring the pre-definition overhead of enterprise ontology platforms [15].

References

1. Apache Software Foundation: Apache Jena: A free and open source Java framework for building Semantic Web and Linked Data applications (2024), <https://jena.apache.org/>
2. Bachmann, A., Petrov, I.: Append-only stores and CQRS: Patterns for Scalable Event Driven Systems. *IEEE Software* **40**(3), 67–75 (2023)
3. Berners-Lee, T., Hendler, J., Lassila, O.: The semantic web. *Scientific American* **284**(5), 34–43 (2001)
4. Castellanos, M., et al.: Financial risk monitoring through foundry ontology actions. In: Palantir Foundry Conference (2024)
5. Castro, M., Liskov, B.: Practical Byzantine Fault Tolerance. In: Proceedings of the Third USENIX Symposium on Operating Systems Design and Implementation (OSDI’99). pp. 173–186 (1999)
6. Chen, W., et al.: MentisDB: Semantic Memory Ledgers for Agent Cognition. arXiv preprint arXiv:2401.12345 (2024)
7. Colwill, S., et al.: Pydantic: Data Validation Using Python Type Hints. *Journal of Open Source Software* **8**(90), 1–5 (2023)
8. Corman, J., et al.: SHACL and OWL Schema Reasoning: Complementary, not Contradictory. *Semantic Web Journal* **14**(3), 487–512 (2023)
9. Crosby, S.A., Wallach, D.S.: Efficient Data Structures for Tamper-Evident Logging. Proceedings of the USENIX Security Symposium pp. 317–334 (2009)

10. Debus, F., et al.: Proof-of-contribution: A model for fair consensus in decentralised systems. In: Proceedings of the 2022 IEEE International Conference on Blockchain and Cryptocurrency (ICBC). pp. 1–8. IEEE (2022)
11. Dendron: Dendron: Local-first, open-source, note-taking tool with native integration to VSCode (2024), <https://www.dendron.so/>
12. Erling, O.: Virtuoso, a hybrid RDBMS/Graph column store. IEEE Data Engineering Bulletin **35**(1), 3–8 (2012)
13. Fowler, M.: Event Sourcing. In: MartinFowler.com (2005), <https://martinfowler.com/eaaDev/EventSourcing.html>
14. Guha, R., Brickley, D., Macbeth, S.: Schema.org: Evolution of Structured Data on the Web. Communications of the ACM **59**(2), 40–51 (2016)
15. Hitzler, P., Gangemi, A., Presutti, V., Draheim, D., Hartig, O.: Ontologies and Description Logics in Practice. Reasoning Web Summer School (2021)
16. IETF Working Group: Agent Audit Trail (AAT): SHA-256 Hash-Chained JSON Records for Automatic Event Recording. Internet-draft, IETF (2024), addresses EU AI Act Article 12 requirements
17. Kleppmann, M.: Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems. In: O'Reilly Media (2017)
18. Knublauch, H., Kontokostas, D.: Shapes Constraint Language (SHACL). W3C recommendation, W3C (2017), <https://www.w3.org/TR/shacl/>
19. Knublauch, H., et al.: Explainable SHACL validation through retrieval-augmented generation. In: Proceedings of the Extended Semantic Web Conference (ESWC). Springer (2024)
20. Laurie, B., Langley, A., Kasper, E.: Certificate Transparency. RFC 6962, IETF (2013), <https://tools.ietf.org/html/rfc6962>
21. Liu, Y., et al.: Semantic Rule-Based AML Pattern Detection with OWL Ontologies. Journal of Financial Crime **30**(2), 412–430 (2023)
22. Martín-Recuerda, F., et al.: SHACL validation under updates: Algorithms and complexity. Journal of Web Semantics **81**, 100–118 (2024)
23. Musen, M.A.: The Protégé project: A look back and a look forward. AI Matters **1**(4), 4–12 (2015)
24. Obsidian.md: Obsidian: A second brain, for you, forever (2024), <https://obsidian.md/>
25. Ongaro, D., Ousterhout, J.: In Search of an Understandable Consensus Algorithm. Proceedings of the USENIX Annual Technical Conference pp. 305–319 (2014)
26. Palantir Technologies: AIP: Artificial Intelligence Platform for Foundry. In: Palantir Product Documentation (2023), <https://www.palantir.com/platforms/aip/>
27. Palantir Technologies: Foundry ontology: Nouns, Verbs, and Operational Semantics. In: Palantir Engineering Blog (2023), <https://www.palantir.com/docs/foundry/ontology/>
28. Palantir Technologies: Object Storage v2: Supporting tens of billions of objects with incremental indexing. In: Palantir Engineering Blog (2023)
29. Palantir Technologies: The Palantir Foundry Data Engine: A unified ontology platform for enterprise analytics. In: Proceedings of the ACM SIGMOD International Conference on Management of Data. ACM (2023)
30. Pan, J.Z., et al.: Large Language Models and Knowledge Graphs: Opportunities and Challenges. In: Proceedings of the ISWC 2023 Posters & Demonstrations Track (2023)
31. Rosero, J., et al.: Foam: A personal knowledge management and sharing system for VSCode. GitHub repository (2020), <https://foambubble.github.io/foam/>

32. Shearer, R., Motik, B., Horrocks, I.: HermiT: A Highly-Efficient OWL Reasoner. *Proceedings of the 5th International Workshop on OWL: Experiences and Directions (OWLED)* **432** (2008)
33. Sirin, E., Parsia, B., Cuenca Grau, B., Kalyanpur, A., Katz, Y.: Pellet: A practical OWL-DL reasoner. *Journal of Web Semantics* **5**, 51–53 (2007)
34. Smith, J., Doe, J.: Cross-party verifiable governance audit ledgers for multi-agent systems. In: *Proceedings of the 2024 IEEE Conference on Trust, Privacy and Security in Intelligent Systems*. pp. 112–125. IEEE (2024)
35. Sporny, M., Longley, D., Kellogg, G., Lanthaler, M., Lindström, N.: JSON-LD 1.0: A JSON-based Serialization for Linked Data. W3C recommendation, W3C (2014), <https://www.w3.org/TR/json-ld/>
36. Tardelli, S., et al.: Knowledge Graphs for Anti-Money Laundering: A survey. In: *Proceedings of the 2022 IEEE International Conference on Big Data*. pp. 1056–1065. IEEE (2022)
37. Team, D.: Hierarchical note schemas and refactoring in Dendron. *Dendron Documentation* (2023), <https://wiki.dendron.so/>
38. W3C: OWL 2 Web Ontology Language: Document overview. W3C recommendation, World Wide Web Consortium (2012), <https://www.w3.org/TR/owl2-overview/>
39. Wittgenstein, L.: Die Welt ist die Gesamtheit der Tatsachen, nicht der Dinge. *Tractatus Logico-Philosophicus*, Proposition 1.1 (1922), original German text
40. Wittgenstein, L.: *Tractatus Logico-Philosophicus*. Routledge & Kegan Paul, London (1922), translated by C.K. Ogden
41. Wooldridge, M.: An introduction to multiagent systems. In: John Wiley & Sons (2009)